Aurora Siger – Comunicação interplanetária da colônia

Sistema de Comunicação Interplanetária da Colônia (SCIC)

A Missão Aurora Siger – Comunicação Interplanetária apresenta o desenvolvimento do Sistema de Comunicação Interplanetária da Colônia (SCIC), um protótipo em Python para organizar, analisar e priorizar os dados operacionais e de comunicação de uma colônia espacial fictícia. O projeto utiliza uma base simulada em CSV, tratada com Pandas, para calcular indicadores, medir o erro absoluto e relativo entre a latência prevista e a observada e estimar a latência com um modelo de regressão linear, avaliado com MAE, MSE, RMSE e R^2. A solução também aplica estruturas de dados estudadas na fase, com um heap para priorizar os alertas mais críticos e uma trie para buscar módulos e sensores por prefixo. Por fim, o sistema relaciona dispositivos de entrada e saída, bases numéricas e eletricidade básica à comunicação da colônia e discute gerenciamento inteligente, sustentabilidade e responsabilidade social, sempre com validação humana nas decisões.



Link do Repositório Público do GitHub: https://github.com/Miguelgonn/Aurora_Comunicacao

Integrantes:

Ana Gabriela rm571312

Kaique rm570533

Miguel Antunes rm573643

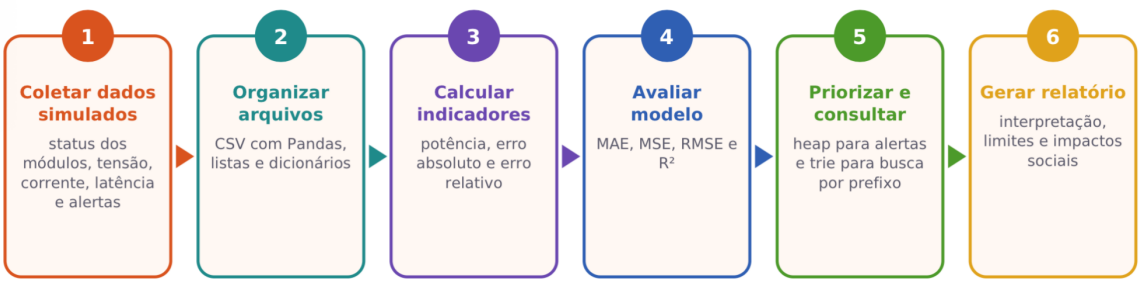
Miguel Gonçalves rm573793

Introdução

Descrição do projeto

A Aurora Siger é uma colônia fictícia formada por módulos de habitação, agricultura, comunicação, comando, energia, laboratório, suporte médico, armazenamento de dados e sensores externos. Todos dependem de uma comunicação interplanetária com latência alta e variável: quando a latência real se afasta da prevista, decisões operacionais podem ser tomadas com informação defasada. O SCIC organiza esses dados, mede o quanto a previsão erra, estima a latência com um modelo simples, decide quais alertas tratar primeiro e permite consultas rápidas por prefixo. O sistema roda no terminal, com um menu de 7 opções (python codigo_fonte.py).

Fluxo do Protótipo



> **Por que isso importa?** Em uma colônia isolada, a equipe humana não consegue olhar todos os alertas ao mesmo tempo. O SCIC transforma dados brutos em uma **ordem de atendimento explicável**, e a decisão final continua sendo das pessoas.

Os dados simulados são carregados de um arquivo CSV e organizados com Pandas (etapas 1 e 2). Em seguida, o sistema calcula os indicadores e os erros e treina o modelo de previsão, avaliado com MAE, MSE, RMSE e R^2 (etapas 3 e 4). Os alertas são ordenados por um heap e os registros são consultados por prefixo com uma trie (etapa 5). Por fim, os resultados são reunidos em uma análise final, disponível na opção 7 do menu (etapa 6).

Objetivos

Objetivo Geral

Desenvolver um protótipo em Python que organize os dados operacionais e de comunicação da Aurora Siger, avalie a confiabilidade de previsões simples, priorize alertas críticos, permita consultas eficientes e gere uma análise técnica de apoio à decisão.

Objetivos Específicos

- Criar e carregar uma base simulada em CSV e tratá-la com Pandas;
- Calcular erro absoluto e erro relativo entre latência prevista e observada, e discutir quando o erro é aceitável;
- Construir um modelo de regressão linear e avaliá-lo com MAE, MSE, RMSE e R^2;
- Priorizar alertas com uma fila de prioridade (heap) implementada manualmente;
- Buscar módulos, tipos e sensores por prefixo com uma trie;
- Relacionar o sistema a dispositivos de E/S, bases numéricas e eletricidade básica;
- Discutir gerenciamento inteligente da comunicação e o impacto social, cultural e sustentável da solução.

Divisão do Projeto: O que é cada fase

O sistema é dividido em módulos de código, e cada opção do menu demonstra um conceito estudado na fase.

Menu	Parte do sistema	O que faz	Conceito	Principais funções
1	Registros	Carrega o CSV, limpa nulos e duplicados, mostra os registros e conta por status	Pandas	carregar_dados()
2	Indicadores e erros	Calcula latência média, potência média, erro absoluto e relativo por módulo	Análise numérica, ponto flutuante	calcular_erros(), indicadores()
3	Modelo e métricas	Estima a latência observada e avalia o desempenho no teste	Regressão linear, treino/teste, MAE, MSE, RMSE, R^2	treinar_modelo()
4	Alertas (heap)	Pontua os alertas e retira sempre o mais urgente	Max-heap, heapify-up e heapify-down	HeapAlertas, pontuacao_alerta()
5	Busca (trie)	Encontra módulos, tipos e sensores pelo prefixo digitado	Trie	Trie, montar_trie()
6	Dispositivos e eletricidade	Converte o código do sensor e calcula potência e resistência	E/S, hex/bin/dec, $P = V \cdot I$, Lei de Ohm	converter_bases(), calcular_eletrica()
7	Análise final	Resume alertas, maior erro, alerta mais urgente e desempenho do modelo	Síntese dos resultados	opcao_analise_final()

Arquivos da entrega (.zip)

Arquivo	Conteúdo
codigo_fonte.py	Sistema completo em Python, com comentários e menu no terminal
dados_aurora_siger.csv	Base simulada com 30 registros (10 módulos × 3 ciclos)
relatorio_tecnico.md	Explicação técnica completa, com análise de erros, modelo, heap, trie, reflexão social e limitações
README.md	Objetivo, arquivos, dependências e modo de execução
link_video.txt	Link do vídeo de apresentação (YouTube, não listado)
graficos_ou_imagens/	Gráficos gerados pela execução (opção 3 do menu)

Dados operacionais e de comunicação

A base dados_aurora_siger.csv é simulada, gerada com semente fixa (42) para ser reproduzível. Se o arquivo não existir, o programa o recria automaticamente. Os dados são lidos com pd.read_csv e passam por limpeza básica (dropna e drop_duplicates)



Campo	Descrição
modulo, tipo	Nome e tipo do módulo (comunicação, habitação, agricultura, laboratório, suporte médico, armazenamento de dados)
codigo_sensor	Código do sensor em hexadecimal (ex.: 0xA001)
ciclo	Ciclo de registro (1 a 3)
tensao_V, corrente_A, potencia_W	Grandezas elétricas do transmissor, com potência = tensão × corrente
carga_pct	Percentual de uso do módulo
latencia_obs_ms, latencia_prev_ms	Latência observada e latência prevista
status	ativo, manutenção ou alerta
essencial, prioridade_base	Se o módulo é essencial (0 ou 1) e prioridade de 1 (baixa) a 5 (crítica)
mensagem	Mensagem resumida do alerta

> **Atenção:** os dados são fictícios. A latência foi construída em função da carga e da potência, com ruído, e algumas **anomalias foram injetadas de propósito** (latência bem acima da prevista) para testar a detecção de alertas.

Indicadores e análise de erros

Para entender a diferença entre o esperado e o observado, o sistema calcula dois erros para cada registro:

```
erro absoluto = | latência observada − latência prevista | (em ms)
erro relativo = erro absoluto / | latência observada | × 100 (em %)
```

O erro relativo permite comparar módulos com escalas de latência diferentes.

Indicador	Valor
Erro absoluto médio	15,32 ms
Erro relativo médio	12,54 %
Erro relativo mediano	8,04 %
Maior erro relativo	42,94 % (Armazenamento de Dados, ciclo 3)
Registros com latência > 125 % da prevista	6

A média (12,54 %) bem acima da mediana (8,04 %) mostra que poucos registros com erro muito grande puxam a média para cima. Os registros em alerta têm erro relativo médio de 34,4 %, contra 7,4 % dos ativos e 4,5 % dos em manutenção.

Quando o erro é aceitável?

Erro relativo	Classificação	Registros
menor que 10 %	Aceitável	19
10 % a 25 %	Atenção	6
maior que 25 %	Preocupante	5

Pequenos desvios são esperados em uma comunicação interplanetária. O erro vira preocupante quando a equipe deixa de confiar na estimativa. Em módulos essenciais, até a faixa de atenção merece acompanhamento.

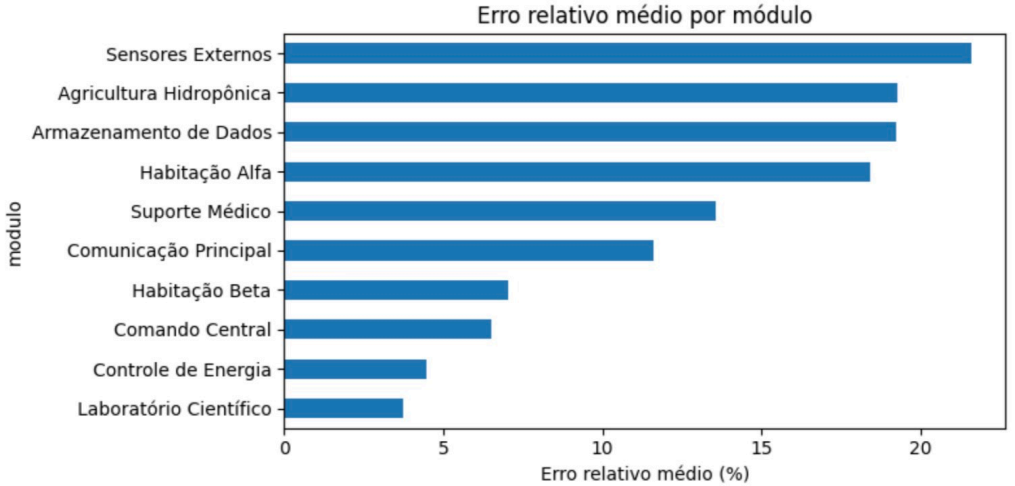


Figura 1 - Erro relativo médio por módulo (gerado pelo sistema)

Leitura conjunta dos dois erros: Armazenamento de Dados tem o maior erro absoluto (31,70 ms), mas Sensores Externos tem o maior erro relativo médio (21,58 %). Por isso os dois indicadores devem ser lidos juntos.

Ponto flutuante e precisão numérica

O computador representa decimais em binário, o que gera pequenas imprecisões.

Em Python, 0.1 + 0.2 resulta em 0.30000000000000004 e 0.1 + 0.2 == 0.3 é False.

Por isso o sistema compara valores com tolerância (np.isclose) e arredonda apenas na exibição.

Pequenas diferenças nos cálculos podem vir dessa aproximação, e não de uma falha real do enlace.

Modelo simples e avaliação de performance

Modelo: regressão linear (scikit-learn) que estima a latência observada a partir da carga (%) e da potência (W). Os dados foram divididos em 70 % para treino e 30 % para teste (random_state=42). Coeficientes: carga = 0,874; potência = −0,106; intercepto = 56,50.

21,13 ms	731,22	27,04 ms	0,38
MAE (erro absoluto médio)	MSE (erro quadrático médio)	RMSE (mesma unidade da variável)	R^2 (explicação do ajuste)

Interpretação das métricas

- **MAE = 21,13 ms:** em média a previsão erra cerca de 21 ms, o que é relevante diante de uma latência média de cerca de 103 ms.
- **RMSE = 27,04 ms:** é um pouco maior que o MAE, mas não passa de 1,3 vez o MAE. Os erros são relativamente homogêneos, sem um pequeno grupo de erros enormes dominando o resultado.
- **R^2 = 0,38:** o modelo explica cerca de 38 % da variação da latência, um desempenho modesto.
- Um único número não basta: um R^2 alto não garantiria um bom modelo, e aqui o R^2 baixo sozinho não diz onde ele falha. Só a leitura conjunta de MAE, RMSE e R^2 mostra que o modelo captura a tendência geral, mas deixa de fora boa parte da variação.

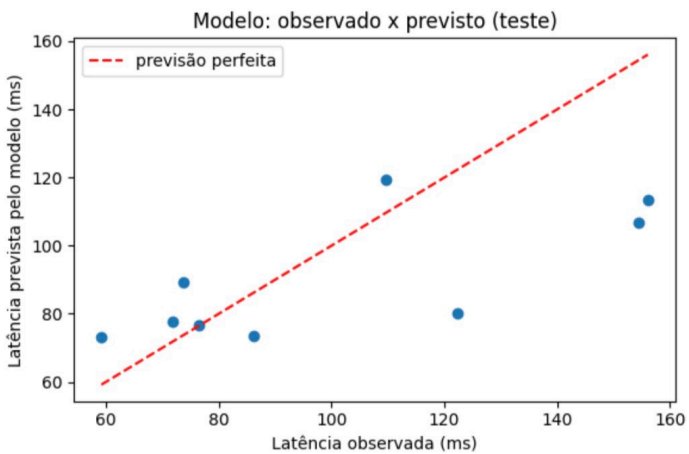


Figura 2 - Latência observada x prevista pelo modelo (conjunto de teste)

Por que o desempenho é modesto?

- A base é pequena (30 registros, cerca de 9 no teste), então as métricas variam bastante conforme a divisão.
- As anomalias injetadas não dependem de carga nem de potência, então a regressão com essas duas variáveis não consegue explicá-las.
- O modelo é intencionalmente simples, como pede a atividade.

> **Resultado honesto:** a estimativa original (`latencia_prev_ms`) teve erro absoluto médio de 15,32 ms, e o modelo teve MAE de 21,13 ms. **O modelo não superou a estimativa existente** neste conjunto, o que indica que seriam necessárias mais variáveis (tráfego, distância do enlace, histórico) e mais dados antes de usá-lo na operação.

Priorização de alertas com heap

Representação: Cada alerta é um dicionário com os dados do registro, guardado em uma tupla (pontuação, alerta). Entram no heap os registros com status = alerta ou erro relativo maior que 25 % (6 alertas na base atual).

Critério de prioridade (quanto maior a pontuação, mais urgente):

```
pontuação = 3 × prioridade_base
+ 5 (se o módulo é essencial)
+ 0,5 × erro relativo (%)
+ 4 (se status = alerta)
```

Estrutura: Um max-heap implementado manualmente (HeapAlertas), com o alerta de maior pontuação na raiz. Na inserção, o item entra no fim e sobe com heapify-up enquanto for maior que o pai. Na remoção do mais urgente, a raiz sai, o último item ocupa seu lugar e desce com heapify-down.

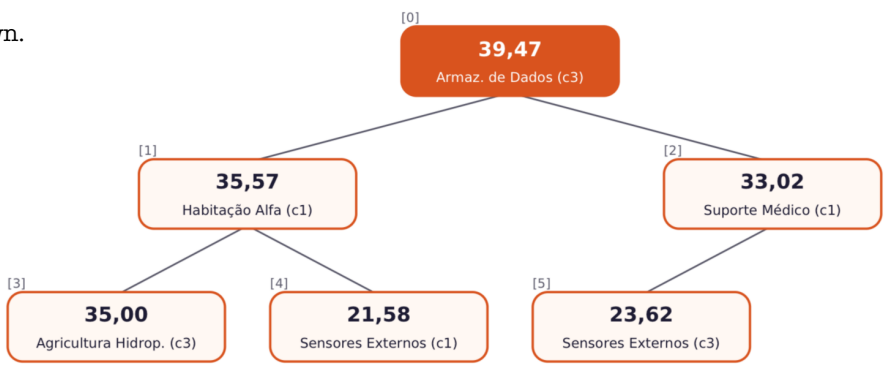


Figura 3 - Heap com os 6 alertas da base atual, em forma de árvore (índice do vetor entre colchetes)

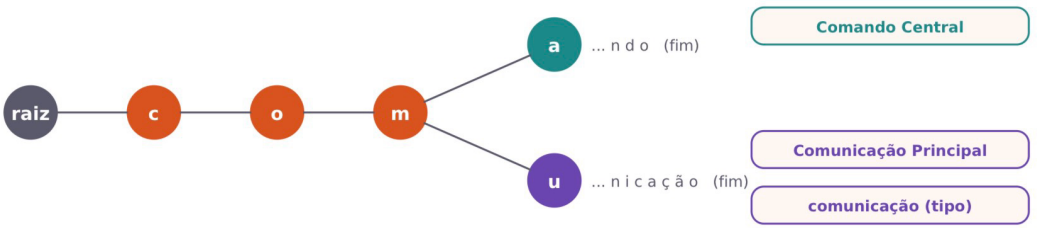
Ordem	Módulo (ciclo)	Pontuação	Erro relativo
1º	Armazenamento de Dados (3)	39,47	42,9 %
2º	Habitação Alfa (1)	35,57	39,1 %
3º	Agricultura Hidropônica (3)	35,00	38,0 %
4º	Suporte Médico (1)	33,02	30,0 %
5º	Sensores Externos (3)	23,62	33,2 %

Armazenamento de Dados lidera por combinar módulo essencial, maior erro relativo e status de alerta. Suporte Médico supera módulos não essenciais com prioridade-base maior por ser essencial.

> **Vantagem sobre uma lista simples:** inserir e retirar o mais urgente custa **O(log n)**. Com uma lista simples seria preciso varrer todos os itens (O(n)) ou reordenar a lista a cada novo alerta (O(n log n)). Em uma colônia onde alertas chegam o tempo todo, isso importa.

Busca de registros com trie

A trie (árvore de prefixos) guarda nomes de módulos, tipos de módulo e códigos de sensores, normalizados em minúsculas e sem acentos, de modo que "com" encontra "Comunicação".



Prefixo digitado: "com" -> desce c > o > m e coleta tudo que está abaixo do nó "m".

Prefixo digitado	Resultados
com	Comando Central, Comunicação Principal, comunicação
hab	Habitação Alfa, Habitação Beta, habitação
0xA0	Os 30 códigos de sensor (0xA001 a 0xA093)

Por que a trie é adequada: a busca percorre só o caminho do prefixo digitado, então o custo depende do **tamanho do prefixo** e não da quantidade de registros. Todos os termos que compartilham o início ficam sob o mesmo nó, o que torna a coleta dos resultados direta, ao contrário de percorrer uma lista inteira comparando texto. Isso favorece consultas rápidas e autocompletar de comandos e códigos.

Dispositivos, bases numéricas e eletricidade

Papel	Dispositivos
Entrada	Sensores e medidores simulados de tensão, corrente e latência (neste protótipo, o arquivo CSV)
Saída	Terminal, relatório técnico e gráficos, que fazem o papel de monitor ou dashboard simulado
Interfaces (conceitual)	Rede e Wi-Fi para telemetria, USB para coleta local, Bluetooth para sensores próximos

Exemplo com o sensor 0xA001 (Comunicação Principal)

Conceito	Cálculo	Resultado
Base numérica	hexadecimal 0xA001 convertido para decimal	40961
Base numérica	0xA001 convertido para binário	1010000000000001
Potência	P = V × I = 12 V × 2,04 A	24,48 W
Lei de Ohm	R = V / I = 12 V / 2,04 A	5,88 Ω

O código hexadecimal serve como identificador compacto do dispositivo, e os endereços seguem um padrão (0xA000 + 16 × índice do módulo + ciclo). A potência entra no modelo como variável.

Gerenciamento inteligente da comunicação

Esta discussão parte dos resultados do SCIC:

- **Sensores e medidores inteligentes:** os campos de tensão, corrente, carga e latência representam o que medidores inteligentes enviariam em tempo real, no lugar do CSV.
- **Monitoramento contínuo:** o erro relativo separa bem o comportamento anormal do normal. Os 6 registros em alerta têm erro médio de 34,4 % contra 7,4 % dos ativos.
- **Automação em situações críticas:** a fila de prioridade indica qual alerta atender primeiro (hoje, Armazenamento de Dados), reduzindo o tempo de decisão. A ação final depende de validação humana.
- **Armazenamento e redundância:** Armazenamento de Dados lidera a fila; se ele falha, perde-se o histórico usado para detectar anomalias. Enlaces redundantes e cópias dos dados reduziriam esse risco.
- **Manutenção preditiva:** módulos na faixa de atenção (Sensores Externos, Agricultura, Armazenamento, Habitação Alfa) são candidatos a inspeção antes de uma falha.
- **Redes inteligentes e microrredes:** a potência média de 56,98 W por transmissor (até 187,20 W) mostra que comunicação e energia estão ligadas. Em uma microrrede, a potência de transmissores ociosos poderia ser reduzida para reservar energia aos módulos essenciais.

Reflexão social, cultural e sustentável

Sustentabilidade: prever bem a latência e priorizar alertas evita retransmissões e gasto desnecessário de energia. Cada watt economizado em uma colônia isolada preserva recursos limitados.

• **Conhecimentos indígenas:** muitas culturas indígenas praticam o uso consciente e cíclico dos recursos, retirando da natureza só o necessário. Essa ideia inspira operar os módulos com folga de capacidade, sem levá-los ao limite de carga.

• **Diversidade e sistemas não enviesados:** a pontuação dá peso fixo a módulos "essenciais", e essa escolha é humana e pode deixar de fora módulos importantes para certos grupos (como a agricultura que sustenta a comunidade). Os pesos devem ser revistos com pessoas de diferentes áreas e culturas.

• **Transparência:** a pontuação é uma fórmula simples e explícita, o que permite explicar por que cada alerta ficou em determinada posição.

• **Responsabilidade humana:** o sistema só recomenda a ordem de atendimento. A equipe humana valida as decisões e pode corrigir o resultado.

• **Linguagem e interpretações justas:** as mensagens de alerta são técnicas e neutras, descrevem o fenômeno ("Latência acima do previsto") e não responsabilizam pessoas ou grupos.

Limitações e possíveis melhorias

Limitações

• Dados simulados e pequenos (30 registros), com anomalias injetadas.

• Modelo linear com duas variáveis, R^2 de 0,38, que não superou a estimativa original.

• Pesos da pontuação definidos pela equipe, sem validação com dados reais.

• Sem sensores, telemetria ou redes reais: os dispositivos são simulados.

Possíveis melhorias

• Ampliar a base e incluir tráfego, distância do enlace e histórico.

• Comparar modelos com AIC/BIC e ajustar hiperparâmetros com Grid ou Random Search.

• Tornar os pesos da pontuação configuráveis e revisá-los com a comunidade.

• Simular com o método de Euler a recuperação da latência após acionar um enlace redundante.

Conclusão

O desenvolvimento do SCIC permitiu reunir, em uma única aplicação, os principais conteúdos estudados na fase: organização de dados com Pandas, análise de erros numéricos, previsão com regressão linear e métricas de desempenho, priorização de alertas com heap, busca por prefixo com trie e a relação com dispositivos, bases numéricas e eletricidade básica. O sistema transforma dados de comunicação da Aurora Siger em informação útil para a decisão, indicando quais módulos merecem atenção primeiro.

Os resultados mostraram que a comunicação da colônia é, em geral, previsível, mas apresenta registros com desvios importantes. O erro relativo médio foi de 12,54%, com mediana de 8,04%, e os registros em alerta chegaram a 34,4% de erro médio. O heap colocou o módulo de Armazenamento de Dados como o alerta mais urgente, por combinar função essencial, maior erro relativo e status de alerta. Já o modelo de regressão apresentou desempenho modesto (R^2 de 0,38) e não superou a estimativa original, o que reforça que um único número não basta para avaliar uma solução e que seriam necessários mais dados e variáveis antes de usá-lo na prática.

Mesmo sendo um protótipo com dados simulados, o SCIC apresenta uma base funcional e transparente para o gerenciamento da comunicação interplanetária. Como evolução, o projeto poderá incluir sensores e monitoramento em tempo real, enlaces redundantes, manutenção preditiva e modelos mais completos. Em todos os casos, a validação humana continua essencial, para que as decisões automatizadas sejam justas, explicáveis e responsáveis.